

2025年4月30日

# Git

東京学芸大学教育学部教育支援課程教育支援専攻 情報教育コース  
進藤修一郎

リポジトリとは？？

# リポジトリとは

## リポジトリとは

過去の変更履歴が記録されている貯蔵庫のこと  
基本的には、変更履歴を保存するディレクトリのことだと思って良い

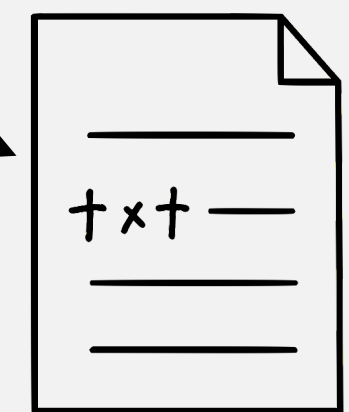
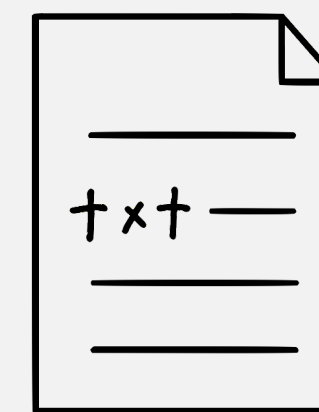
### リポジトリ

stud

- .gitが保存されているディレクトリがリポジトリとなる.
- この場合**study**ディレクトリが、リポジトリである.
- Studyディレクトリ配下にあるディレクトリ、ファイル変更等が、.gitファイルに記録される

English

grammar



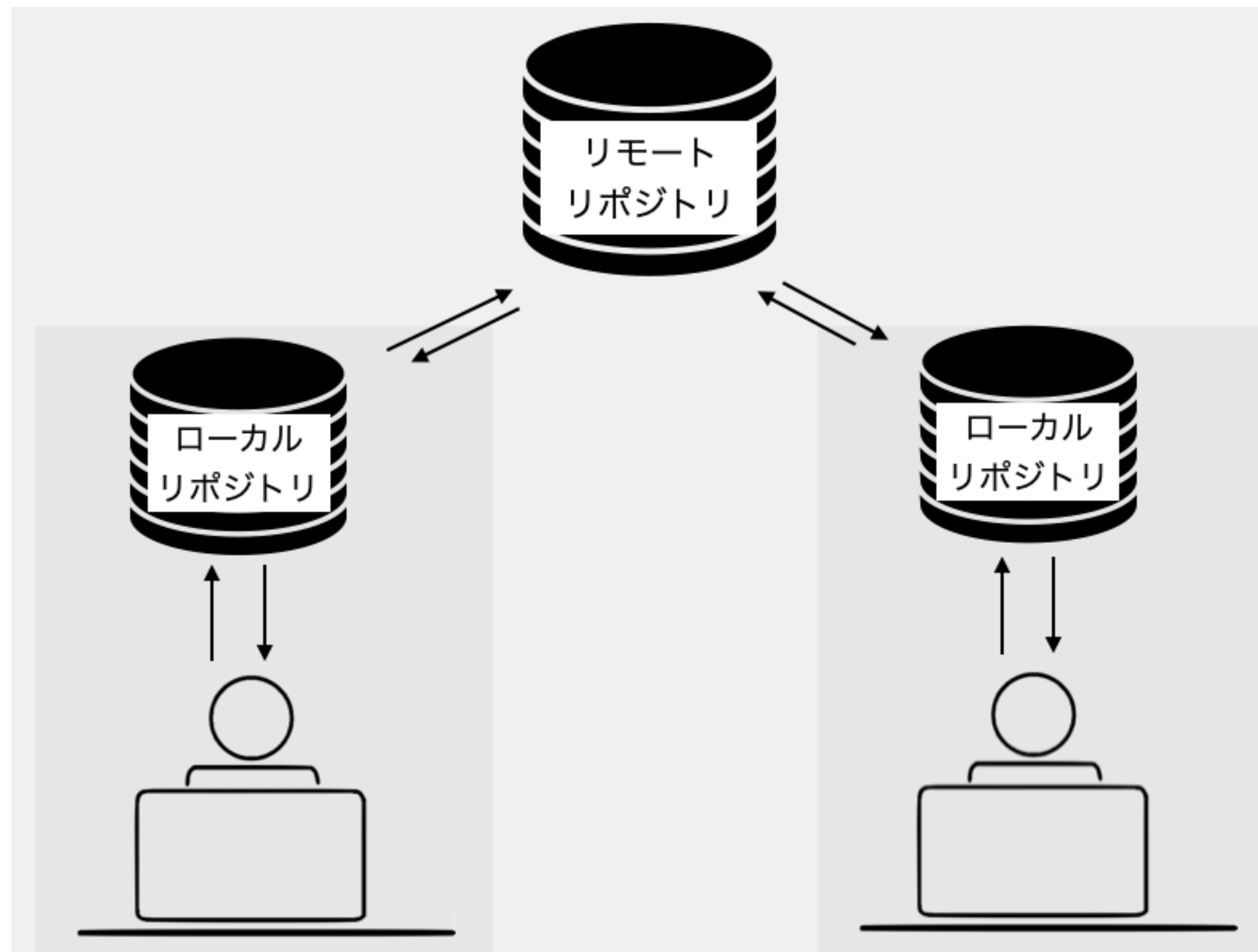
.git

.gitというファイルに変更履歴が保存される.  
先頭に「.」がついているファイルは隠しファイルとなり、実際には表示されない

# リポジトリとは

## リポジトリとは

過去の変更履歴が記録されている貯蔵庫のこと  
リモートリポジトリとローカルリポジトリの2つが存在する



## リモートリポジトリ

サーバーに配置されているリポジトリ

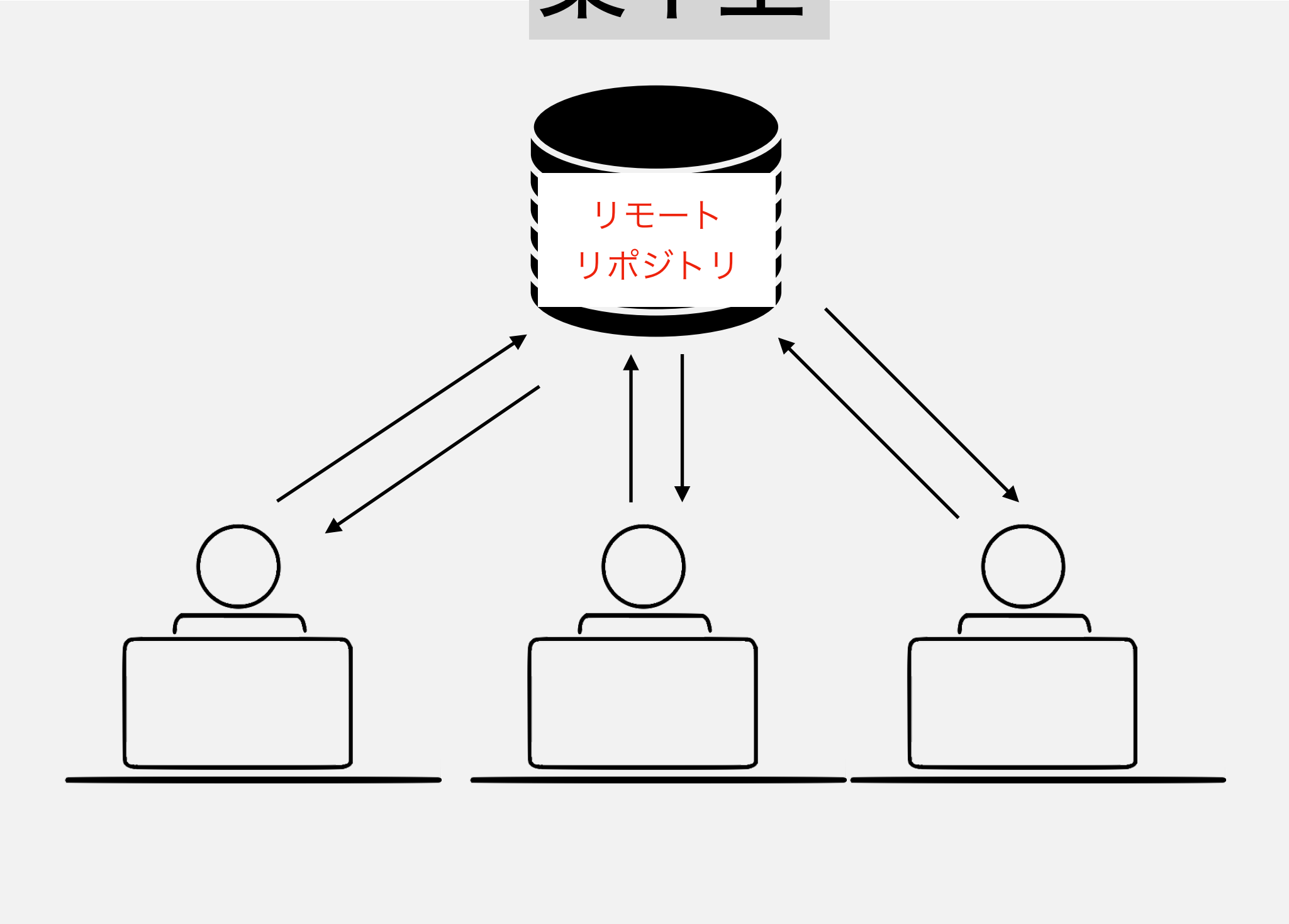
## ローカルリポジトリ

自分のパソコン上のリポジトリ

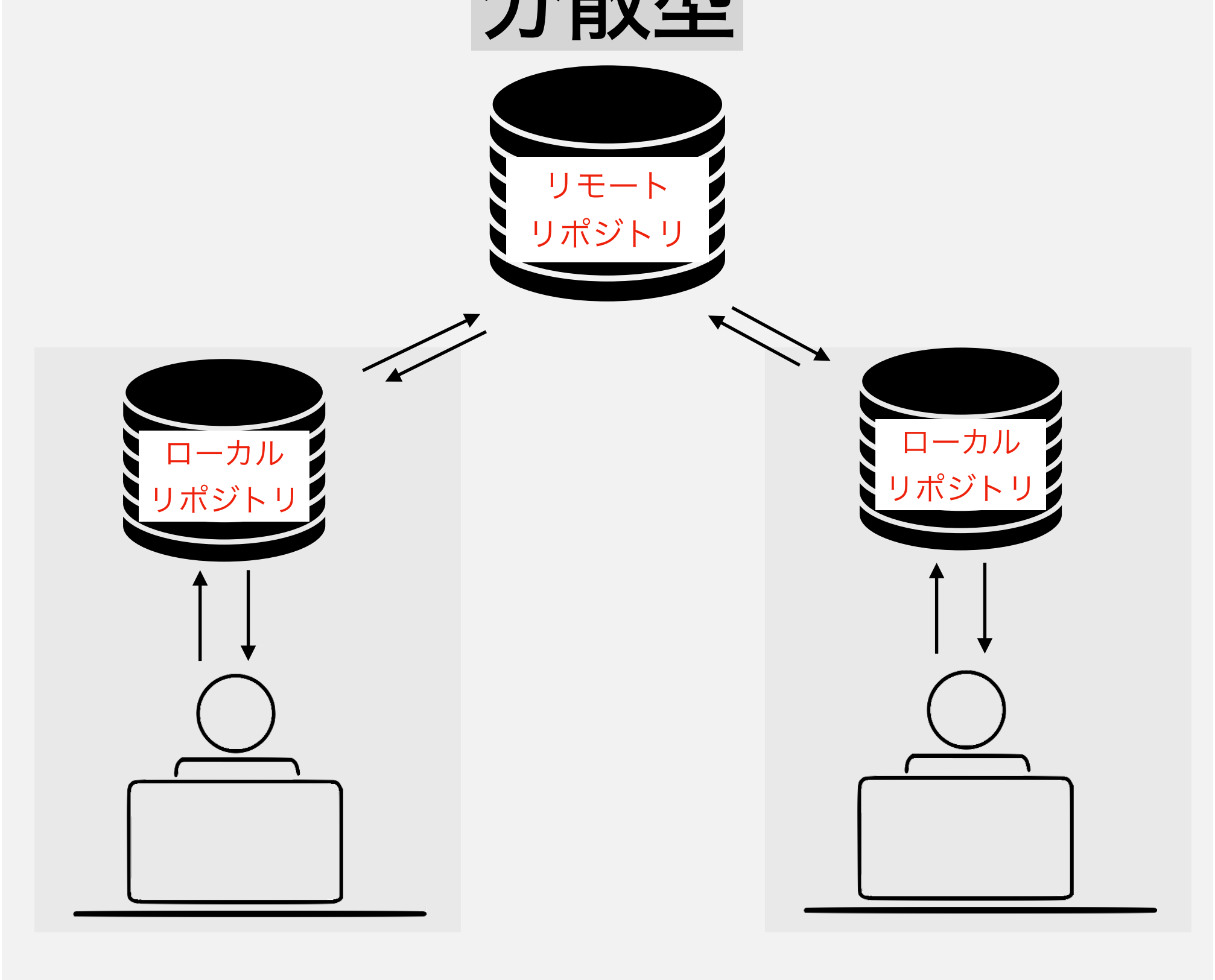
## 分散型と集中型

# 分散型と集中型

## 集中型



## 分散型



|      |                                                                                                   |                                                                                           |
|------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| 特徴   | <ul style="list-style-type: none"><li>ネットワークに接続した状態で操作をする</li><li>全ての変更がリモートリポジトリに反映される</li></ul> | <ul style="list-style-type: none"><li>ネットワークに接続しなくても利用できる</li><li>個人のパソコン内で操作する</li></ul> |
| メリット | <ul style="list-style-type: none"><li>シンプルな構造である</li></ul>                                        | <ul style="list-style-type: none"><li>オフラインでも編集できる</li><li>他の人に影響を及ぼさずに、編集できる</li></ul>   |

# 分散型とバージョン管理

## Gitとは

分散型バージョン管理システムのこと

## 分散型

各自のパソコンにリポジトリをもち  
好きなタイミングで同期をする

## バージョン管理システム

ファイルに対して、誰が、いつ、  
どのような変更したのかの変更履歴  
に関する情報を記録すること

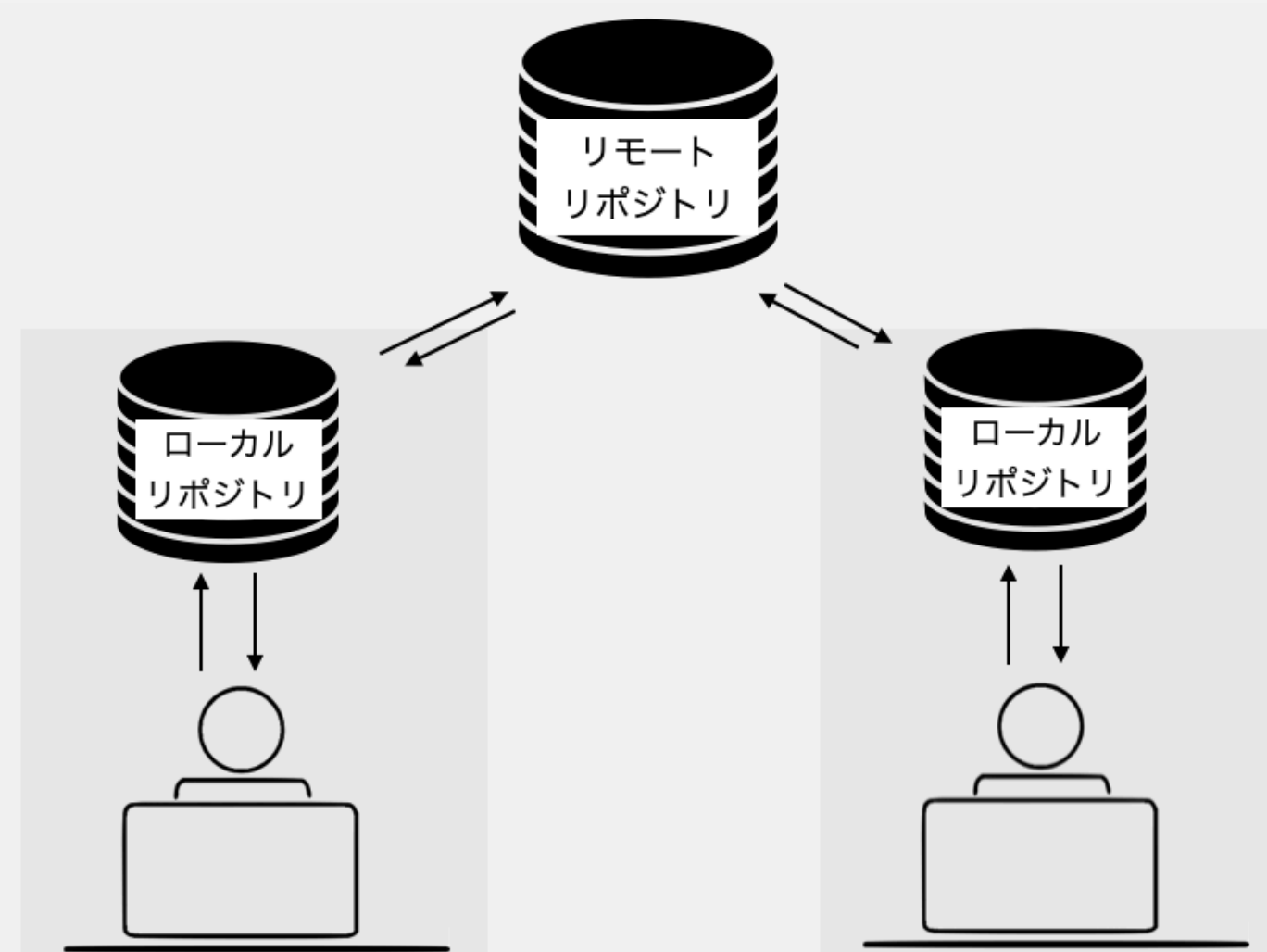
# Gitとは

## Gitとは

分散型バージョン管理システムのこと

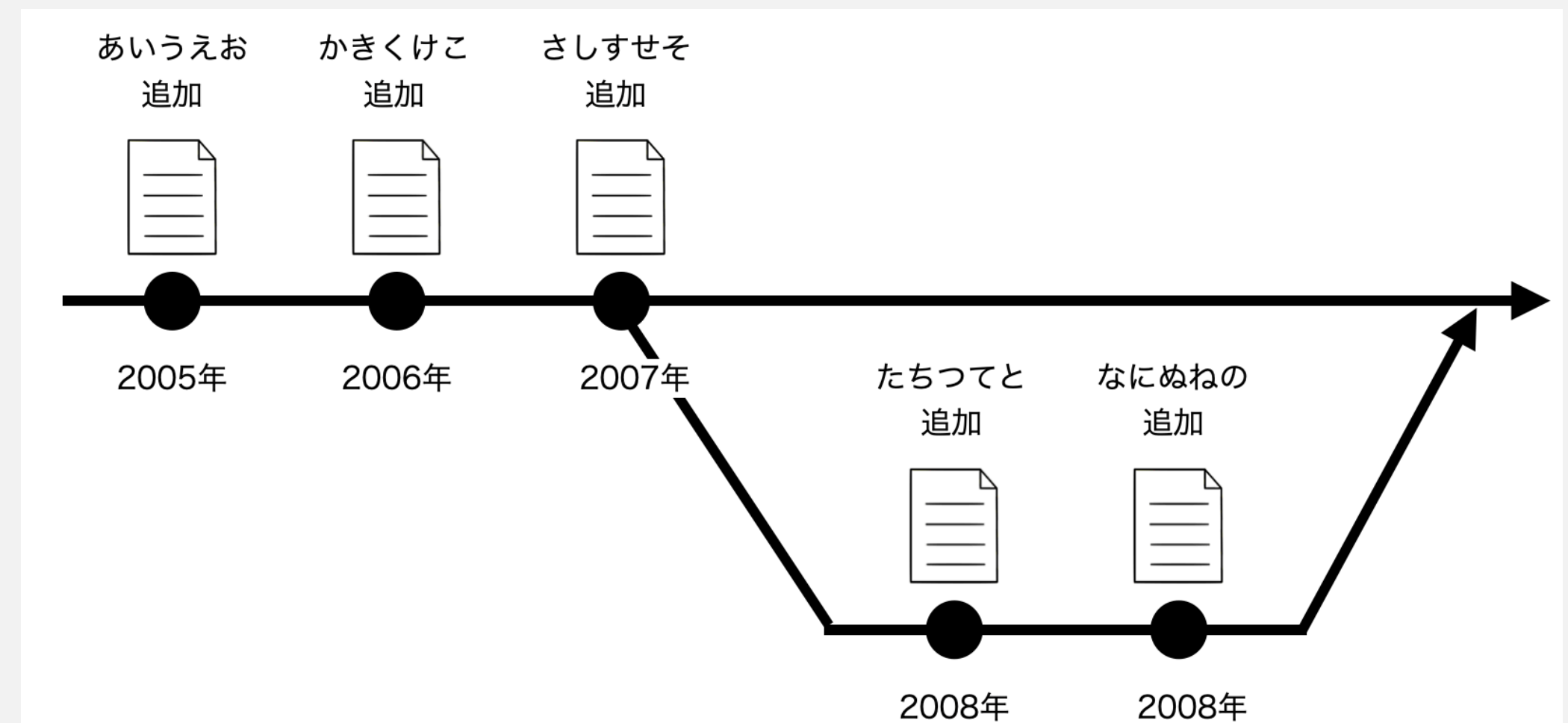
### Gitを利用するメリット①

分散型なのでチームで共有編集や共有開発が効率化  
できる



### Gitを利用するメリット②

バージョン管理システムなので文書を編集し直すときに、  
バックアップを取る必要がなくなる



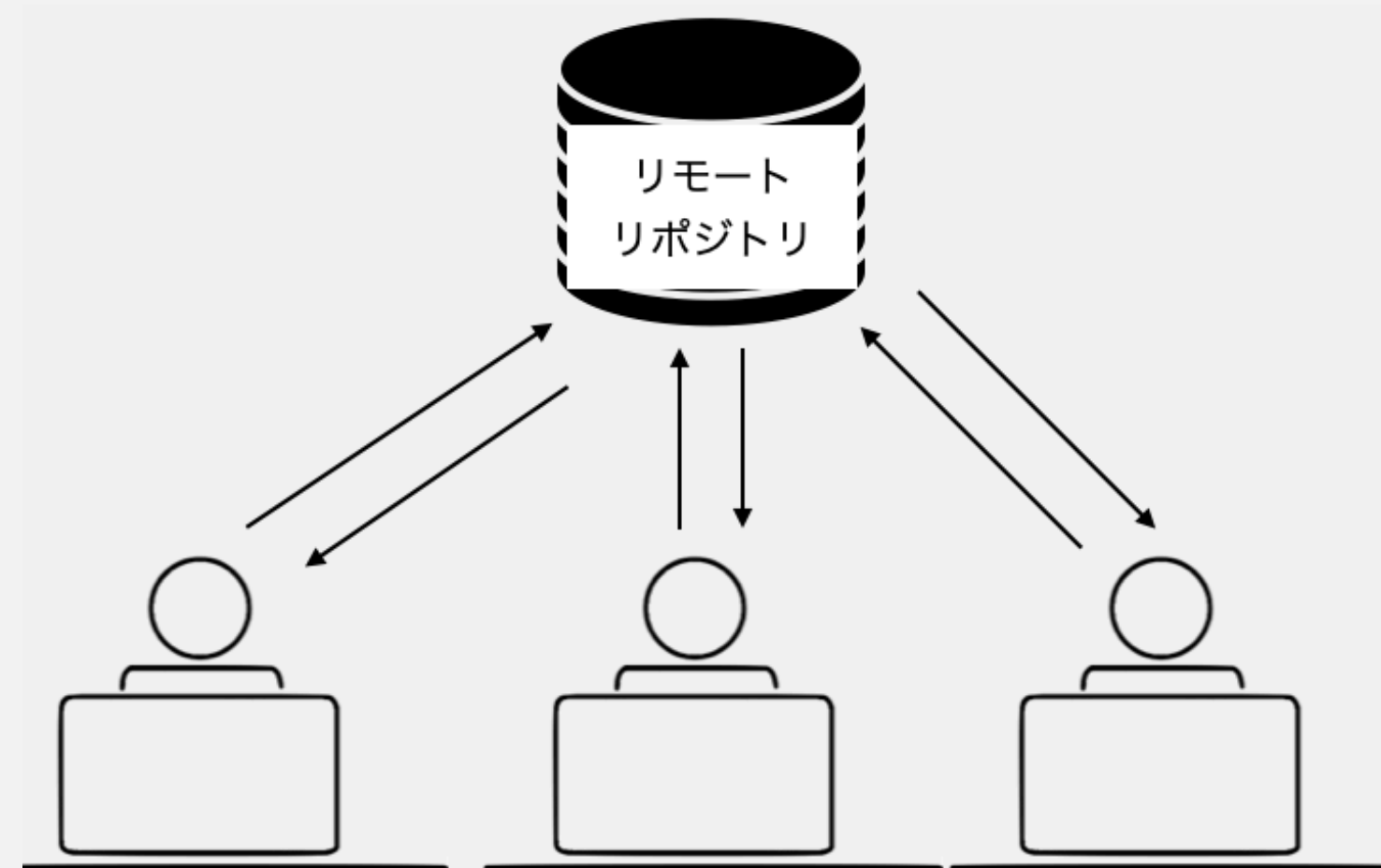
# 分散型について

## 分散型

各自のパソコンにリポジトリをもち好きなタイミングで同期をする

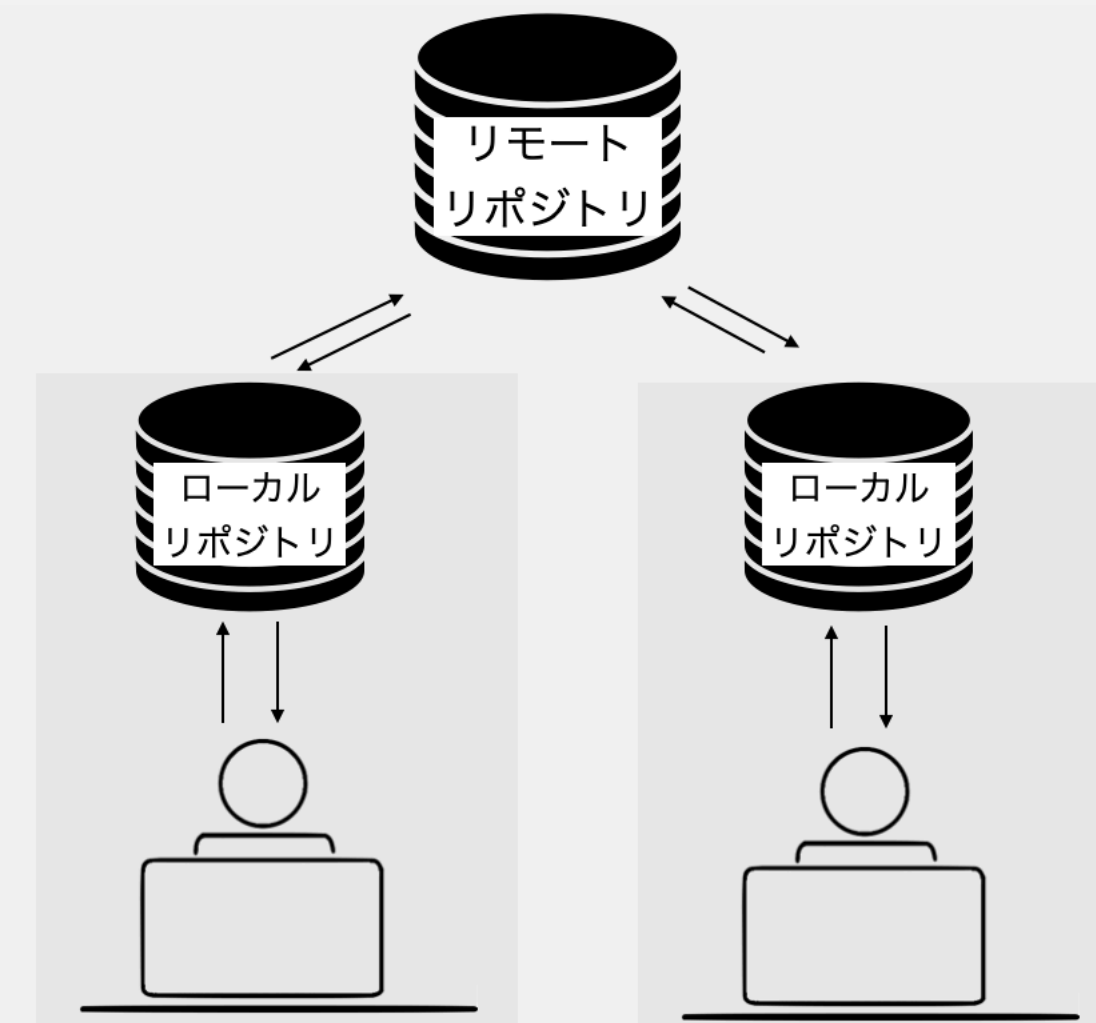
### Gitを利用しないと

- ・ ネットワークに利用していないと利用できない
- ・ 直接リモートリポジトリを編集し，即時反映される



### Gitを利用すれば

- ・ ローカルで編集することができる
- ・ 好きなタイミングで同期をさせられる



# バージョン管理システムについて

## バージョン管理システム

ファイルに対して、誰が、いつ、どのような変更したのかの変更履歴に関する情報を記録すること

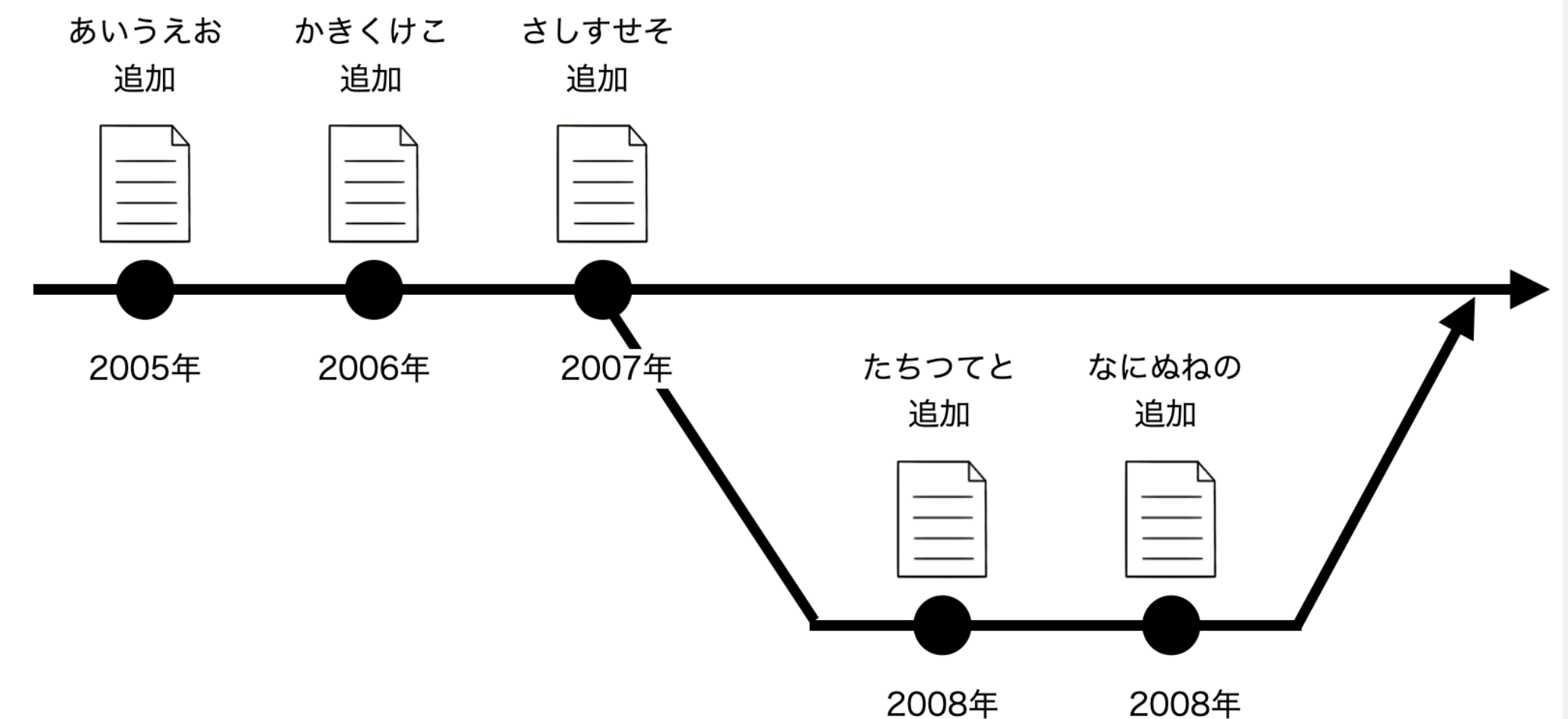
### Gitを利用しないと

- ・新しい編集用のファイルを複製し、そちらを編集する
- ・それによって、安心して編集することができる
- ・しかし、毎回バックアップを取るのは大変...

20250401.pages  
20250401のコピー.pages  
20250401下書き.pages  
20250401下...のコピー.pages  
20250401最新版.pages

### Gitを利用すれば

- ・1つのファイルで変更履歴を保存できる(add,commit)
- ・1つのファイルでバックアップを取りつつ変更ができる(branch,merge)



# 実際にGitを操作してみよう

デスクトップにgit\_practiceというディレクトリを作成する

1. デスクトップで、二本指でクリックをし、新規フォルダを作成を選択
2. ディレクトリ名を「git\_practice」にする
3. ターミナルでgit\_practiceディレクトリに移動する



ターミナルを開く

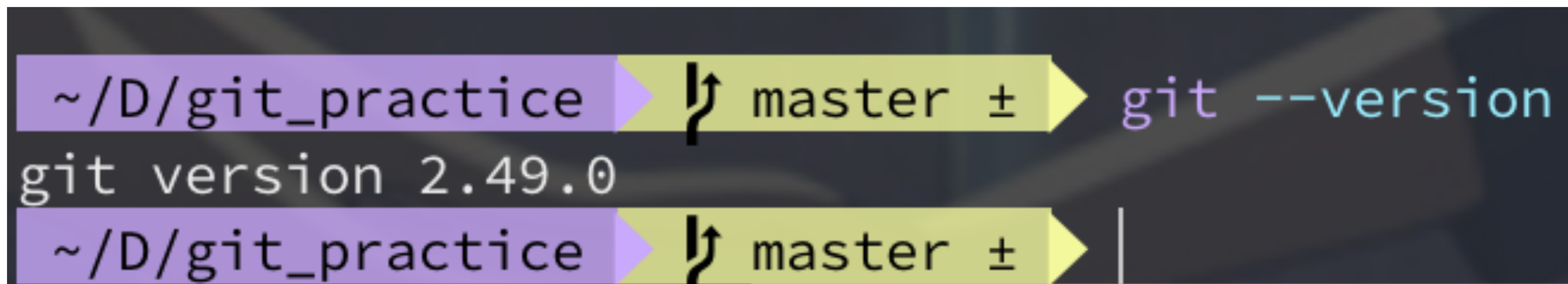
1. デスクトップに移動する
2. 「git\_practice」に移動する

```
~ ➤ cd Desktop/git_practice/  
~/D/git_practice ➤ |
```

## 実際にGitを操作してみよう

Gitがインストールされているのか確かめるコマンド

# git --version



```
~/D/git_practice ➤ master ± git --version  
git version 2.49.0  
~/D/git_practice ➤ master ± |
```

Gitがインストールされていない場合

`brew install git`

とかを入力してあげよう

## 実際にGitを操作してみよう

Gitを利用するには、ユーザー名、メールアドレスを登録する必要がある。

```
git config user.name
```

```
git config user.email
```

でユーザー名、メールアドレスを確認することができる

## 実際にGitを操作してみよう

初めて利用する場合、ユーザー名、メールアドレスは登録されていないので、  
下記のコマンドで登録する必要がある。

`git config user.name “自分の名前”`

`git config user.email "登録したいメールアドレス"`

# 実際にGitを操作してみよう

## Gitコマンドの一覧表示

git

```
~/D/git_practice ➤ git
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
          [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
          [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy
-fetch]
          [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
          [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envva
r>]
          <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  restore    Restore working tree files
  rm         Remove files from the working tree and from the index
```

## 実際にGitを操作してみよう

変更履歴を保存するファイルを作成する

# git init

```
~/D/git_practice ➤ master ➤ ls -a  
./  ../  .git/
```

# git initコマンド

リポジトリ

git\_pactice

diary.txt

.git

.gitというファイルに変更履歴が保存される.  
先頭に「.」がついているファイルは隠しファイル  
となり、実際には表示されない

git init コマンドにより、この隠しファイル  
を作成した

実際に編集するファイルを作成する

# touch diary.txt



# Gitの3つのエリア

## Gitの3つのエリア

Gitには、ワーキングエリア、ステージングエリア、リポジトリの3つのエリアがある

### 作業エリア

作業中のファイル

ステージング

`git add ファイル名`

### ステージングエリア

開発者がファイル全体ではなく、指定した部分だけをコミットできるようにするGitの機能

### コミット履歴

コミット

`git commit -m “変更内容”`

確定した変更履歴が保存される場所

ファイルをステージングする

# git add ファイル名

```
~/D/git_practice ➤ master ± git add diary.txt  
~/D/git_practice ➤ master ± |
```

## 実際にGitを操作してみよう

ファイルをコミットする

# git commit -m "変更内容"

```
~/D/git_practice ➤ master ± git commit -m "2010年の日記を追加"  
[master 57f985f] 2010年の日記を追加  
Committer: Shuichi Kudo <shuichiroshindo@MacBook-Pro.local>  
Your name and email address were configured automatically based  
on your username and hostname. Please check that they are accurate.  
You can suppress this message by setting them explicitly:  
  
git config --global user.name "Your Name"  
git config --global user.email you@example.com  
  
After doing this, you may fix the identity used for this commit with:  
  
git commit --amend --reset-author  
  
1 file changed, 4 deletions(-)
```

# 実際にGitを操作してみよう

コミット履歴を確認

## git log

```
commit 03c8ecafbc60a3ea0e32d2c83816239352a2d484 (HEAD -> master)
Author: Shuichi Kudo <shuichiroshindo@MacBook-Pro.local>
Date:   Mon Apr 28 15:14:05 2025 +0900
```

2010年の日記を追加

# 実際にGitを操作してみよう

## (演習1) ファイルの編集, ステージング, コミットの練習

現在作成したdiary.txtで下記の操作をしよう

1. 「2011年に記載しました」という文章を先ほどの文章の下に追記
2. diary.txtをステージング
3. 「2011年の日記を追加」というコメントでdiary.txtをコミット
4. 「2012年に記載しました」という文章を先ほどの文章の下に追記
5. diary.txtをステージング
6. 「2012年の日記を追加」というコメントでdiary.txtをコミット
7. 変更履歴を確認する

## 実際にGitを操作してみよう

1. 「2011年に記載しました」という文章を先ほどの文章の下に追記
2. diary.txtをステージング

**git add diary.txt**

3. 「2011年の日記を追加」というコメントでdiary.txtをコミット

**git commit -m "2011年の日記を追加"**

4. 「2012年に記載しました」という文章を先ほどの文章の下に追記
5. diary.txtをステージング

**git add diary.txt**

6. 「2012年の日記を追加」というコメントでdiary.txtをコミット

**git add diary.txt**

7. 変更履歴を確認する

**git commit -m "2012年の日記を追加"**

# 実際にGitを操作してみよう

```
~/D/git_practice ➤ master ± ➤ git log
commit f2591c05c6ae3ddf49fcf3f46393a606514d6946 (HEAD -> master)
Author: Shuichi Kudo <shuichiroshindo@MacBook-Pro.local>
Date: Mon Apr 28 15:36:48 2025 +0900

    2012年の日記を追加
    y.txt

commit 3b59870ba40abc2341830e516226efcab2b5178
Author: Shuichi Kudo <shuichiroshindo@MacBook-Pro.local>
Date: Mon Apr 28 15:36:17 2025 +0900

    2011年の日記を追加
    ...

commit 03c8ecafbc60a3ea0e32d2c83816239352a2d484
Author: Shuichi Kudo <shuichiroshindo@MacBook-Pro.local>
Date: Mon Apr 28 15:14:05 2025 +0900

    2010年の日記を追加
```

## 実際にGitを操作してみよう

過去の状態に戻る

# git checkout コミットID

```
commit 03c8ecafbc60a3ea0e32d2c83816239352a2d484 (HEAD -> master)
```

```
Author: Shuichi Kudo <shuichiroshindo@MacBook-Pro.local>
```

```
Date: Mon Apr 28 15:14:05 2025 +0900
```

2010年の日記を追加

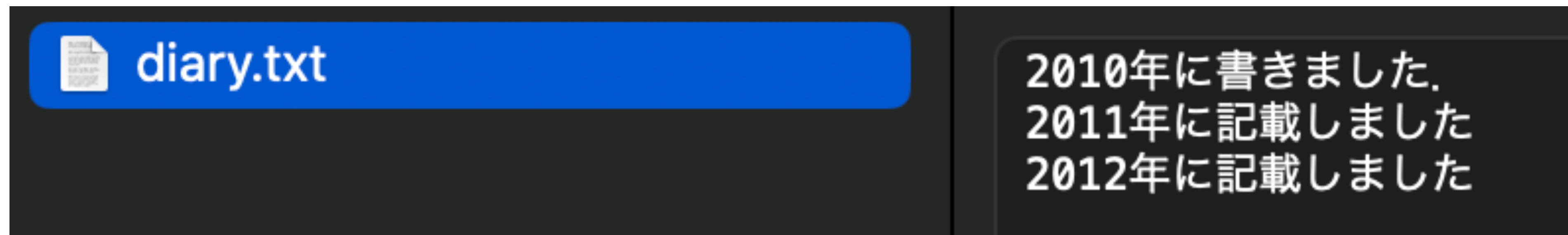
```
~/D/git_practice master ± git checkout 03c8ecafbc60a3ea0e32d2c83816239352  
a2d484
```

diary.txt

2010年に書きました。

最新の状態に戻る

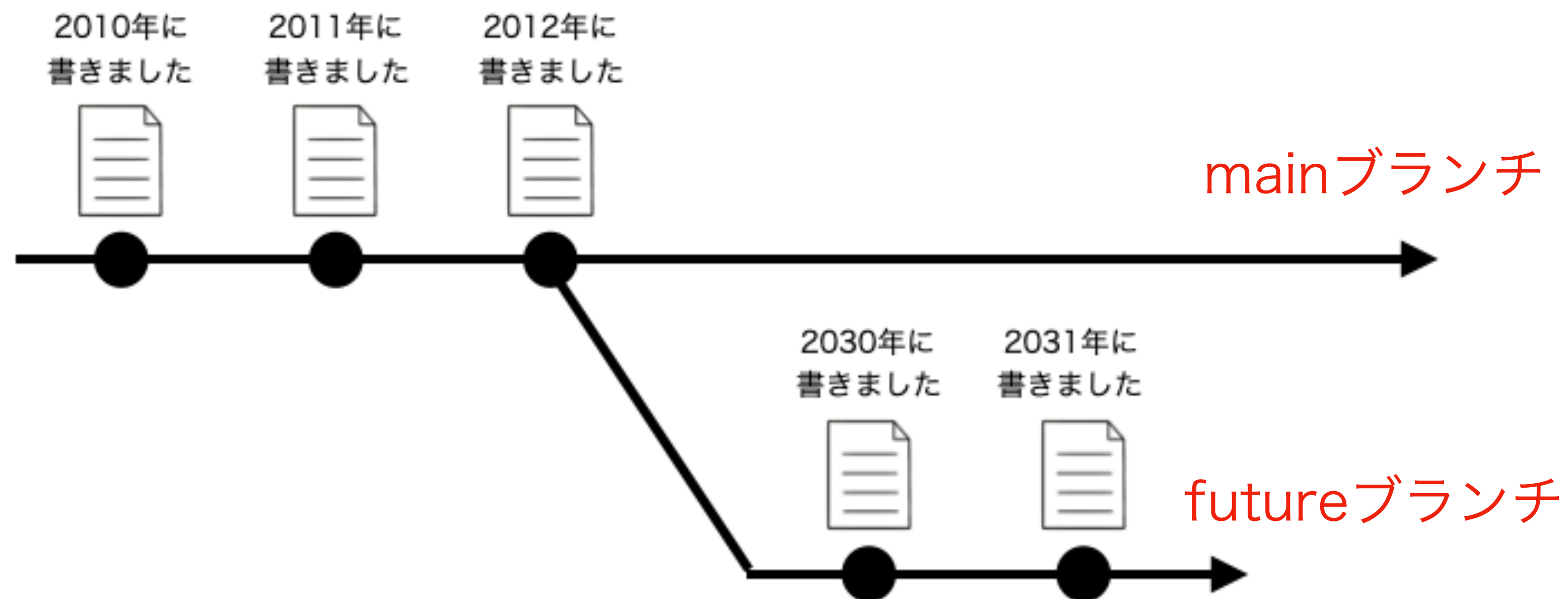
# git checkout master



# ブランチとは

## ブランチ

ブランチ (branch) は、開発における経路のひとつを意味する。リポジトリのコピーを作り、そのコピーで作業できる。ブランチを切る基準は、組織やプロジェクトによるが、通常は、新しい機能を作ったり、コードを修正したりするときにブランチを切ることが多い。

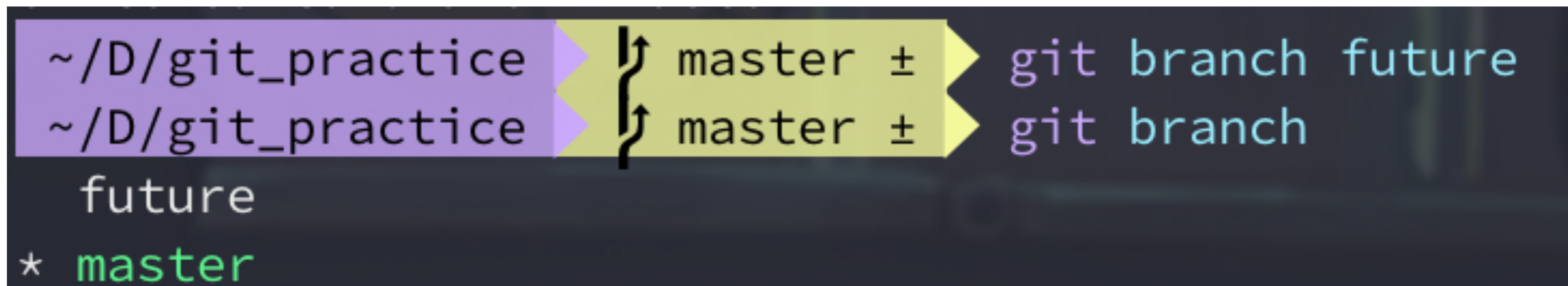


現在のブランチの確認

# git branch

## ブランチの作成

# git branch ブランチ名



The image shows a terminal window with a dark background. The first two lines show the current directory as ~/D/git\_practice. The third line shows the output of 'git branch', which is 'future'. The fourth line shows the output of 'git branch', which is 'master'. The 'master' branch is marked with an asterisk, indicating it is the current branch. The text is color-coded: the directory path is purple, the branch names are green, and the asterisk is red.

```
~/D/git_practice  
~/D/git_practice  
future  
* master
```

branchの切り替え

# git checkout ブランチ名

```
~/D/git_practice ➤ ↶ master ± ➤ git checkout future
Switched to branch 'future'
~/D/git_practice ➤ ↶ future ± ➤ git branch
* future
master
```

## 実際にGitを操作してみよう

(演習2)futureブランチでのファイルの編集, ステージング, コミットの練習

現在作成したdiary.txtで下記の操作をしよう

1. 「2030年に記載しました」という文章を先ほどの文章の下に追記
2. diary.txtをステージング
3. 「2030年の日記を追加（futureブランチ）」というコメントでdiary.txtをコミット
4. 「2031年に記載しました」という文章を先ほどの文章の下に追記
5. diary.txtをステージング
6. 「2031年の日記を追加（futureブランチ）」というコメントでdiary.txtをコミット

## 実際にGitを操作してみよう

現在作成したdiary.txtで下記の操作をしよう

1. 「2030年に記載しました」という文章を先ほどの文章の下に追記
2. diary.txtをステージング

**git add diary.txt**

3. 「2030年の日記を追加（futureブランチ）」というコメントでdiary.txtをコミット

**git commit -m "2030年の日記を追加（futureブランチ）"**

4. 「2031年に記載しました」という文章を先ほどの文章の下に追記
5. diary.txtをステージング

**git add diary.txt**

6. 「2031年の日記を追加（futureブランチ）」というコメントでdiary.txtをコミット

**git commit -m "2031年の日記を追加（futureブランチ）"**

## 実際にGitを操作してみよう

mainブランチに戻る

# git checkout master

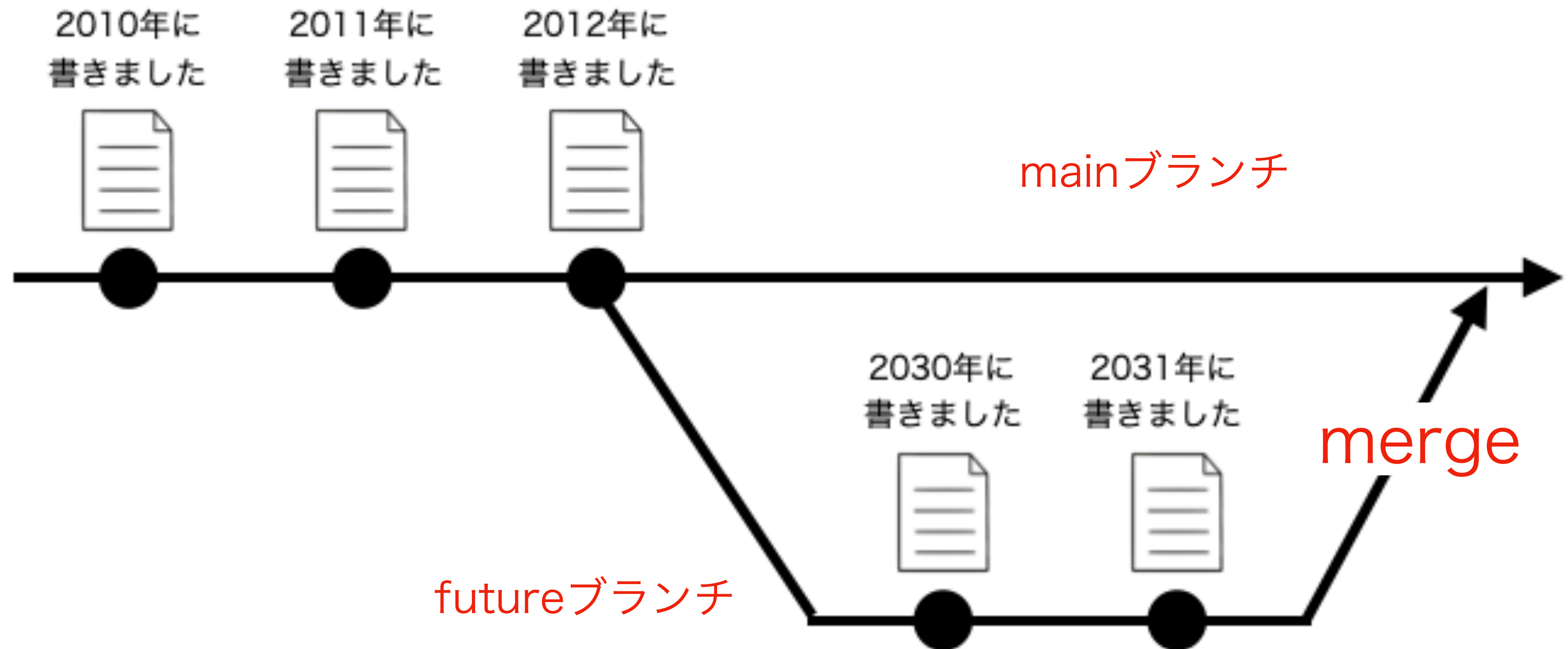
でmainブランチに戻ることができる. futureブランチでファイルを編集し、コミットしたため、mainブランチのファイルは書き換えられていな

```
Switched to branch 'future'
~/D/git_practice ➤ ↶ future ± ➤ git checkout master
Switched to branch 'master'
~/D/git_practice ➤ ↶ master ± ➤ |
```

# mergeとは

## merge

作成したブランチの内容をmainブランチに統合することをmergeという



ブランチをマージ

# git merge ブランチ名

```
~/D/git_practice ➤ master ± ➤ git merge future  
  
Updating f2591c0..fd26771  
Fast-forward  
 diary.txt | 4 +++-  
1 file changed, 3 insertions(+), 1 deletion(-)  
~/D/git_practice ➤ master ± ➤
```

| diary.txt                                                                    |
|------------------------------------------------------------------------------|
| 2010年に書きました。<br>2011年に記載しました<br>2012年に記載しました<br>2030年に記載しました<br>2031年に記載しました |

ブランチを削除する

# git branch -d ブランチ名

```
~/D/git_practice ➤ ↗ master ± ➤ git branch
future
* master
~/D/git_practice ➤ ↗ master ± ➤ git branch -d future
Deleted branch future (was fd26771).
~/D/git_practice ➤ ↗ master ± ➤ git branch
* master
```

(演習3)future2ブランチを作成し， マージし， 削除する

現在作成したdiary.txtで下記の操作をしよう

1. future2ブランチを作成する
2. future2ブランチに移動する
3. future2ブランチで， 「2050年に書きました」という文章をdiary.txtに追記する
4. diary.txtをステージングする
5. 「2050年の日記を追加（future2ブランチ）」という文章でdiary.txtをコミットする
6. mainブランチに移動する
7. mainブランチでfuture2ブランチをmergeする
8. future2ブランチを削除する

# 実際にGitを操作してみよう

1. future2ブランチを作成する

**git branch future2**

2. Future2ブランチに移動する

**git checkout future2**

3. future2ブランチで, 「2050年に書きました」という文章をdiary.txtに追記する

4. Diary.txtをステージングする

**git add diary.txt**

5. 「2050年の日記を追加 (future2ブランチ)」という文章でdiary.txtをコミットする

**git commit -m "2050年の日記を追加 (future2ブランチ)"**

6. mainブランチに移動する

**git checkout master**

7. mainブランチでfuture2ブランチをmergeする

**git merge future2**

8. future2ブランチを削除する

**git branch -d future2**

# コンフリクトとは

## コンフリクト

Gitで管理する複数のデータに、重複や競合が発生している状態のこと

# 実際にGitを操作してみよう

## (演習4)

1. 「git\_practice」ディレクトリにweather.txtを作成する
2. weather.txtの1行目に「今日の天気：晴れでした。」という文章を記述する
3. weather.txtをステージング
4. weather.txtを「最初の天気レポートを追加」という文章でコミット
5. mainブランチから、「sunny」と「cloudy」という新たな2つのブランチを作成
6. sunnyとcloudyができているか確認
7. sunnyブランチに切り替える
8. sunnyブランチで、weather.txtの2行目に「今日の天気：とても晴れていて、気持ちよかった」と追記
9. ステージングして「sunny: 晴れの詳細を追加」という文章でコミット
10. cloudyブランチに切り替える
11. cloudyブランチで、weather.txtの2行目に「今日の天気：曇りで少し肌寒かったです」と追記
12. ステージングして「cloudy: 曇りの詳細を追加」という文章でコミット
13. メインブランチに戻る

# 実際にGitを操作してみよう

1. 「git\_practice」ディレクトリにweather.txtを作成する  
`touch weather.txt`
2. Weather.txtの1行目に「今日の天気：晴れでした。」という文章を記述する
3. weather.txtをステージング  
`git add weather.txt`
4. Weather.txtを「最初の天気レポートを追加」という文章でコミット  
`git commit -m "最初の天気レポートを追加"`
5. Mainブランチから、「sunny」と「cloudy」という新たな2つのブランチを作成  
`git branch sunny`  
`git branch cloudy`
6. sunnyとcloudyができているか確認  
`git branch`
7. sunnyブランチに切り替える  
`git checkout sunny`
8. sunnyブランチで、weather.txtの2行目に「今日の天気：とても晴れていて、気持ちよかった」と追記
9. ステージングして「sunny: 晴れの詳細を追加」という文章でコミット  
`git add weather.txt`  
`git commit -m "sunny: 晴れの詳細を追加"`
10. cloudyブランチに切り替える  
`git checkout cloudy`
11. cloudyブランチで、weather.txtの2行目に「今日の天気：曇りで少し肌寒かったです」と追記
12. ステージングして「cloudy: 曇りの詳細を追加」という文章でコミット  
`git add weather.txt`  
`git commit -m "cloudy: 曇りの詳細を追加"`
13. メインブランチに戻る  
`git checkout main`

## 実際にGitを操作してみよう

実際にコンフリクトを発生させる

# git merge sunny

```
~/D/git_practice ➤ master ± ➤ git merge sunny  
  
Updating f8eeb1b..fbd2a7b  
Fast-forward  
  weather.txt | 3 ++-  
1 file changed, 2 insertions(+), 1 deletion(-)
```

## 実際にGitを操作してみよう

実際にコンフリクトを発生させる

# git merge cloudy

```
1 file changed, 2 insertions(+), 1 deletion(-)
~/D/git_practice ➤ ↶ master ± ➤ git merge cloudy
Auto-merging weather.txt
CONFLICT (content): Merge conflict in weather.txt
Automatic merge failed; fix conflicts and then commit the result.
✗ ➤ ~/D/git_practice ➤ ↶ master ± ➤ |
```

diary.txt

weather.txt

```
今日の天気：晴れでした.
<<<<<< HEAD
今日の天気：とても晴れていて、気持ちよか
った
=====
今日の天気：曇りで少し肌寒かったです
>>>>>> cloudy
```

## 実際にGitを操作してみよう

2行目の部分を手動で修正し，その後，ステージング，コミットをする．

`git add weather.txt`

`git commit -m "コンフリクトを解消"`

```
Automatic merge failed; fix conflicts and then commit the result.
~/D/git_practice master ± git add weather.txt
                             git commit -m "コンフリクトを解消"
                             git_practice

[master 6071497] コンフリクトを解消
Committer: Shuichi Kudo <shuichiroshindo@MacBook-Pro.local>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly:

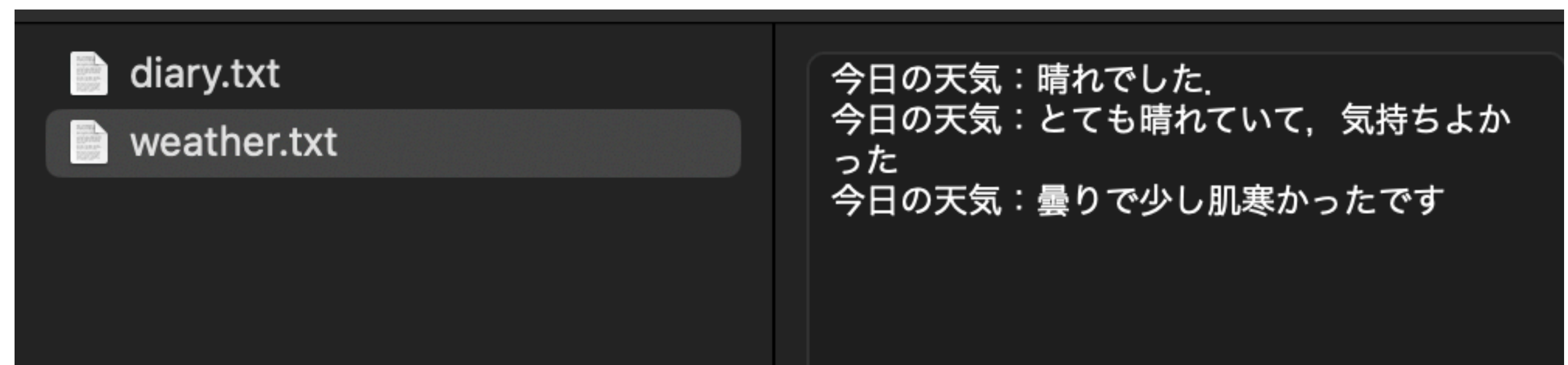
    git config --global user.name "Your Name"
    git config --global user.email you@example.com

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

~/D/git_practice master ± git log
commit 6071497d2800769cc8cfed0ea9348f6915f3b785 (HEAD -> master)
Merge: fbd2a7b 611ecba
Author: Shuichi Kudo <shuichiroshindo@MacBook-Pro.local>
Date: Mon Apr 28 17:30:49 2025 +0900

    コンフリクトを解消
```



## 実際にGitを操作してみよう

Gitがインストールされているのか確かめるコマンド

`git --version`

## 実際にGitを操作してみよう

Gitがインストールされているのか確かめるコマンド

`git --version`